PROG5 (GBIN5U03) - Examen de première session
Année 2025-2026

UFR IM²AG

UNIVERSITÉ Grenoble Alpes

Durée : 2h.
Tout document ou calculatrice interdit. Un dictionnaire bilingue non annoté pour les étudiants étrangers. Pour chaque question, une partie des points peut tenir compte de la présentation.
Le barême est indicatif.

1 Ligne (8 points)

Dans cette partie, nous nous intéressons aux fonctions `gets` et `fgets`. La fonction `gets` permet de lire une ligne (tous les caractères jusqu'au premier `'\n'` ou bien jusqu'à la fin de fichier) sur l'entrée standard (clavier) et place le résultat en mémoire à l'adresse `str` donnée en paramètre. Le caractère `'\n'`, s'il est présent, n'est pas conservé, et un `'\0'` est ajouté en fin de chaîne. Cette fonction renvoie NULL si elle n'a lu aucun caractère (fin de fichier), dans ce cas la mémoire pointée par `str` reste inchangée. Dans le cas contraire, elle renvoie le pointeur qui lui est passé en paramètre. Son prototype est le suivant :

```
char *gets(char *str);
```

La fonction `fgets` fait la même chose que `gets`, elle lit une ligne, avec les différences suivantes : elle lit les caractères depuis un flux passé en paramètre, elle lit au plus `size-1` caractères depuis ce flux et elle conserve le caractère `'\n'` s'il est présent. Elle renvoie NULL si elle n'a lu aucun caractère et dans ce cas la mémoire pointée par `str` reste inchangée. Son prototype est :

```
char *fgets(char *str, int size, FILE *stream);
```

Questions :

1. (2 point) si on suppose que le flux d'entrée utilisé est `stdin` et que le caractère `'\n'` final ne nous intéresse pas, peut-on écrire `fgets` en utilisant `gets` ? `gets` en utilisant `fgets` ? justifiez vos réponses soit en expliquant pourquoi c'est impossible, soit en écrivant la fonction répondant à la question.

2. (1 points) pourquoi est-il fortement déconseillé d'utiliser la fonction `gets` bien qu'elle fasse partie de la bibliothèque standard du C ?

3. (2 points) La fonction `fgetc` permet de lire un caractère sur un flux donné. Elle renvoie le caractère lu en cas de succès ou la constante EOF en cas d'erreur ou de fin de fichier rencontrée. Son prototype est le suivant :
   ```
   int fgetc(FILE *stream);
   ```
   Écrivez en C la fonction `fgets` en utilisant `fgetc`.

4. (3 points) écrivez en C une fonction `fgetline` permettant de lire une ligne depuis un flux et retournant un pointeur sur `char` correspondant au début de la chaîne lue. Son prototype sera le suivant :
   ```
   char *fgetline(FILE *stream);
   ```
   Comme dans le cas de `gets` cette fonction n'aura pas de taille maximale pour la chaine lue: elle ne s'arêtera qu'en cas d'erreur, de fin de fichier rencontrée, ou au premier `'\n'` (qui devra

être inclus à la ligne avant le `'\0'` terminal). En revanche, cette fonction devra allouer elle même la mémoire dans laquelle la chaîne lue sera stockée. En cas d'erreur (de lecture ou d'allocation), elle devra retourner `NULL` et s'assurer qu'elle ne laisse aucune fuite mémoire.

2 Tri (3 points)

La bibliothèque standard du C contient plusieurs fonctions de tri (`qsort`, `heapsort`, `mergesort`) dont les noms sont tirés des algorithmes qu'elles mettent en œuvre. Toutes ces fonctions prennent le même type de paramètres : l'adresse de début du tableau à trier `base`, le nombre d'éléments du tableau `nmemb`, la taille en octets d'un élément `size` et une fonction de comparaison `compar`. A titre d'exemple, le prototype de la fonction `qsort` de la bibliothèque C standard est le suivant :
`void qsort(void *base, size_t nmemb, size_t size, int (*compar)(void *, void *));`

Question :

1. écrivez en C une fonction `montri` permettant de trier un tableau d'éléments (algorithme au choix) et ayant le même prototype que la fonction `qsort`. Bien entendu, l'utilisation d'une des fonctions de tri de la bibliothèque standard est interdite. En revanche, si cela vous est utile, vous pouvez utiliser la fonction `memex` suivante :
   `void memex(void *a, void *b, size_t taille);`
   qui échange le contenu des `taille` octets situés en `a` avec celui des `taille` octets situés en `b`.

   La fonction `montri` à écrire pourra s'utiliser de la manière suivante :

```c
#include "tri.h"
#include <stdio.h>
#include <string.h>

int compare(void *a, void *b) {
        char *ca = (char *) a;
        char *cb = (char *) b;
        return *ca - *cb;
}

int main() {
        char essai[]="salut, c'est moi";

        montri(essai, strlen(essai), sizeof(char), compare);
        printf("%s\n",essai);
        return 0;
}
```

   Le résultat affiché est alors :
   `',aceilmossttu`

3 Sort (9 points)

La commande UNIX `sort` permet de trier les lignes d'un texte selon l'ordre lexicographique (ordre du dictionnaire). Voici un extrait de sa page de manuel :

```
SORT(1)                  Manuel de l utilisateur Linux                  SORT(1)

NOM
       sort - Trier les lignes d'un fichier texte.

SYNOPSIS
       sort [fichier...]

DESCRIPTION
       sort trie, [...] toutes les lignes des fichiers indiqués.
       Si  aucun fichier n'est fourni, ou si le nom `-' est mentionné, la lec-
       ture se fera depuis l'entrée standard.

       Par défaut, sort écrit ses résultats sur la sortie standard.
```

Voici un petit exemple d'utilisation de cette commande :

```
> cat Mon_fichier
essai
prévu
pour bien montrer l'
exécution
de sort
> sort Mon_fichier
de sort
essai
exécution
pour bien montrer l'
prévu
```

ATTENTION : les questions suivantes vous demandent de construire progressivement un seul et même programme. Si vous le désirez, vous pouvez toutes les traiter ensemble. Dans tous les cas la concision du code, en particulier sa factorisation, sera prise en compte dans l'évaluation.

Questions :

1. (5 points) écrivez en C un programme ayant le même comportement que la commande `sort`. Si cela vous aide, vous pourrez supposer que le nombre de lignes lues par votre commande est toujours inférieur ou égal à 1024. Toutes les fonctions mentionnées dans les parties 1 et 2 pourront être utilisées.

   La fonction `strcmp` de la bibliothèque standard pourra également être utilisée, son prototype est :

`int strcmp(const char *s1, const char *s2);`
étant donné deux chaines de caractères `s1` et `s2` cette fonction renvoie respectivement un entier positif, nul ou négatif selon si `s1` est respectivement plus grande, égale ou inférieure à `s2`.

2. (1 points) si vous avez supposé que le nombre de lignes lues par votre programme est inférieur ou égal à 1024, proposez une modification de votre code permettant de lever cette hypothèse. Si votre programme fonctionne déjà correctement sans cette hypothèse, les points de cette question vous seront automatiquement crédités.

3. (3 points) modifiez votre programme pour qu'il accepte les options suivantes :

   - `-r` : trie les lignes dans l'ordre inverse
   - `-o nom` : écrit la sortie dans le fichier nommé `nom` plutôt que sur la sortie standard
   - `-u` : n'affiche qu'une seule fois les lignes identiques